

Algoritmos y Arquitecturas Computacionales de Alto Rendimiento

2026-2 | Ejercicios de práctica | Primera parte

Temas: estructuras de decisión y estructuras de repetición

Los enunciados aquí presentes se deben resolver implementando programas en Python. El código fuente debe estar comentado, explicando el funcionamiento del programa.

Ejercicio 1. Calculadora de funciones trigonométricas por series

Construir una calculadora de funciones trigonométricas que permita seleccionar una función a través de un menú, por ejemplo:

1. Seno
2. Coseno
3. Tangente
4. Salir

Al ingresar un valor de x en grados, el programa debe obtener el valor de la función para ese valor de x , pero usando la fórmula de serie infinita de la función seleccionada. Para el cálculo se utilizan n términos de la serie, obteniendo así un valor aproximado de la función. Las series que representan al seno y al coseno son:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$
$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n}$$

Recuerde que hay que cambiar el valor de infinito en el límite de la serie por un entero finito, y que el argumento x debe convertirse de grados a radianes antes de evaluar la serie.

Para el caso de la tangente no hay que implementarla, pero el programa debe mostrar un mensaje indicando que con esa opción se podría hallar, pero que aún no ha sido implementada. El menú debe repetirse hasta que el usuario seleccione la opción de salir.

Ejercicio 2. Representación de un número en base b

Construir un programa que lea un entero positivo n y luego lea un entero positivo b que debe ser menor que 10. El programa deberá mostrar la representación del número n en base b .

Tenga en cuenta que la base más pequeña que tiene sentido es $b = 2$, de modo que el valor leído debe validarse en el rango $2 \leq b < 10$; si está fuera de ese rango, el programa debe informarlo y solicitarlo nuevamente.

Ejercicio 3. Inversión de una frase

Construir un programa que lea una frase y la invierta palabra a palabra, y también letra a letra. Para este problema consultar la función `split()`.

Ejercicio 4. Intercalado de ceros entre las cifras

Escribir un programa que lea un entero positivo y escriba el mismo número intercalando un "0" entre cada cifra. Por ejemplo: 4567 se convierte en 4050607.

Ejercicio 5. Media de los valores pares

Construir un programa que lea un número variable de valores enteros. El resultado que entregará el programa es la media de los números pares de entre los leídos, es decir, la suma de todos los valores pares dividida por el número de estos. Implementarlo usando listas, y también sin usarlas.

- a) Defina y documente el criterio con el que termina la lectura de datos (por ejemplo, un valor centinela o una cantidad indicada previamente por el usuario).
- b) Considere explícitamente el caso en que no se lea ningún valor par.

Ejercicio 6. Desviación estándar

¿Es posible construir programas similares para la desviación estándar? Si lo es, constrúyalo(s).

- a) Indique si calcula la desviación estándar poblacional o la muestral, y señale en qué se diferencian.
- b) Analice qué dificultad adicional aparece en la versión que no utiliza listas y cómo la resuelve.

Ejercicio 7. Moda de un conjunto de enteros

Construir un programa que calcule la moda de una lista de elementos enteros. El programa debe indicar de manera explícita qué hace cuando el conjunto es multimodal, es decir, cuando hay más de un valor con la misma frecuencia máxima, y qué hace cuando ningún valor se repite.

Ejercicio 8. Clasificación de triángulos

Construir un programa que lea las longitudes de los tres lados de un triángulo y determine, en primer lugar, si esos tres valores pueden formar realmente un triángulo, aplicando la desigualdad triangular. Si es así, el programa debe clasificarlo:

- a) Por sus lados: equilátero, isósceles o escaleno.
- b) Por sus ángulos: rectángulo, acutángulo u obtusángulo, comparando el cuadrado del lado mayor con la suma de los cuadrados de los otros dos.

El programa debe rechazar longitudes negativas o nulas y repetir el proceso hasta que el usuario ingrese un valor centinela, por ejemplo un 0 como primera longitud.

Ejercicio 9. Aproximación de π con criterio de tolerancia

En el ejercicio 1 el número de términos n se fija de antemano. Escribir ahora un programa que aproxime el valor de π mediante la serie de Leibniz:

$$\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots$$

pero que, en lugar de recibir el número de términos, reciba una tolerancia ε y detenga la suma cuando el valor absoluto del último término calculado sea menor que ε . El programa debe mostrar la aproximación obtenida, el número de términos utilizados y el error respecto al valor de `math.pi`.

Ejecute el programa con $\varepsilon = 10^{-2}$, 10^{-4} y 10^{-6} , y comente qué ocurre con el número de términos necesarios a medida que la tolerancia disminuye.

Ejercicio 10. Números primos menores o iguales que n

Construir un programa que lea un entero positivo n y muestre todos los números primos menores o iguales que n , empleando únicamente estructuras de decisión y repetición, es decir, verificando la divisibilidad por tanteo y sin recurrir a bibliotecas externas. Al terminar, el programa debe indicar cuántos primos encontró y cuál es el mayor de ellos. Como mejora, haga que la búsqueda de divisores de cada número se detenga en su raíz cuadrada y justifique, en los comentarios del código, por qué esa optimización es correcta.